

Predicción de Calorías Quemadas en Sesiones de Entrenamiento Utilizando Aprendizaje de Máquina

Emiliano Camacho Ponce

*Instituto Tecnológico y de Estudios Superiores de Monterrey
Campus Querétaro*

I. INTRODUCCIÓN

El ejercicio físico y el monitoreo de la actividad deportiva se han vuelto cada vez más relevantes gracias a la popularización de dispositivos y aplicaciones de seguimiento de salud, que generan grandes volúmenes de datos sobre el desempeño físico de las personas en cada sesión de entrenamiento. Estimar de forma precisa el gasto calórico de una sesión es útil tanto para el usuario, que busca planear su rutina y sus objetivos de composición corporal, como para entrenadores y aplicaciones de fitness, que buscan personalizar recomendaciones a partir de las características fisiológicas de cada persona.

Tradicionalmente, la estimación de calorías quemadas se ha basado en fórmulas generales que consideran variables como el peso corporal y la duración del ejercicio, sin incorporar de forma conjunta otros factores fisiológicos que también influyen en el gasto energético, como la frecuencia cardíaca o el porcentaje de grasa corporal. Con el avance del aprendizaje de máquina, es posible construir modelos que aprendan patrones a partir de datos reales de entrenamiento para generar estimaciones más ajustadas a las condiciones particulares de cada sesión.

En este trabajo se aborda el problema de predecir la cantidad de calorías quemadas en una sesión de entrenamiento en el gimnasio, utilizando un modelo de regresión lineal y un modelo de Random Forest.

Toda la solución se implementó en Python en dos variantes, una sin utilizar bibliotecas de aprendizaje automático ni de estadística avanzada para el algoritmo en sí, y otra utilizando scikit-learn como apoyo.

II. ETL

A. ¿Qué es ETL?

ETL es un acrónimo de Extracción, Transformación y Carga (Extract, Transform, Load), un proceso fundamental en ciencia de datos que consiste en obtener datos de una fuente, limpiarlos y transformarlos a un formato adecuado, y

finalmente cargarlos para su uso en análisis o modelos de aprendizaje de máquina. Este proceso garantiza que los datos utilizados en el modelado sean consistentes, estén libres de errores evidentes y representen correctamente el fenómeno que se busca modelar.

B. Datos

Para este estudio se utilizó un dataset de miembros de gimnasio con información sobre sus características fisiológicas y el desempeño registrado durante sus sesiones de entrenamiento. El conjunto de datos consta de 973 registros y 15 columnas, donde cada registro representa una sesión de entrenamiento de un miembro, incluyendo datos demográficos, indicadores fisiológicos (frecuencia cardíaca, porcentaje de grasa corporal, índice de masa corporal), características de la sesión (duración, tipo de entrenamiento, frecuencia semanal) y las calorías quemadas durante dicha sesión. A continuación, en la Tabla 1, se presentan las features con su descripción y tipo de dato del dataset completo.

Feature	Descripción	Tipo de dato
Age	Edad del miembro del gimnasio	int
Gender	Género del miembro (Male / Female)	string
Weight (kg)	Peso corporal en kilogramos	float
Height (m)	Estatura en metros	float
Max_BPM	Frecuencia cardíaca máxima registrada durante la sesión	int
Avg_BPM	Frecuencia cardíaca promedio durante la sesión	int
Resting_BPM	Frecuencia cardíaca en reposo	int
Session_Duration (hours)	Duración de la sesión de entrenamiento en horas	float
Calories_Burned	Calorías quemadas durante la sesión	float
Workout_Type	Tipo de entrenamiento (Yoga, string	

	HIIT, Cardio, Strength)	
Fat_Percentage	Porcentaje de grasa corporal	float
Water_Intake (liters)	Ingesta de agua en litros durante la sesión	float
Workout_Frequency (days/week)	Frecuencia de entrenamiento en días por semana	int
Experience_Level	Nivel de experiencia del miembro (escala 1-3)	int
BMI	Índice de masa corporal	float

Tabla 1. Features del dataset completo

El dataset es de dominio público y se puede acceder y consultar a través del siguiente enlace: <https://www.kaggle.com/datasets/valakhorasani/gym-members-exercise-dataset>

C. Desarrollo

1. Extracción

Lo primero que se hizo fue extraer el dataset de Kaggle en el enlace anterior para posteriormente, utilizando la librería de pandas, cargar el archivo CSV en un DataFrame.

2. Transformación

Este dataset no presenta valores faltantes ni registros duplicados en ninguna de sus 973 filas, por lo que no fue necesario eliminar ni imputar registros por esta razón. Sin embargo, sí fue necesario realizar las siguientes transformaciones para dejar los datos en un formato adecuado para el modelado:

Codificación de variables categóricas: los features Gender y Workout_Type son de tipo categórico (string), por lo que se convirtieron a variables numéricas mediante one-hot encoding, generando columnas binarias para cada categoría.

Análisis de correlación: para poder seleccionar las features predictoras, se generó una matriz de correlación con coeficientes de Pearson. Esto con el objetivo de ver qué features tienen una relación lineal más fuerte con la variable objetivo de Calories_Burned. Los resultados fueron los siguientes:

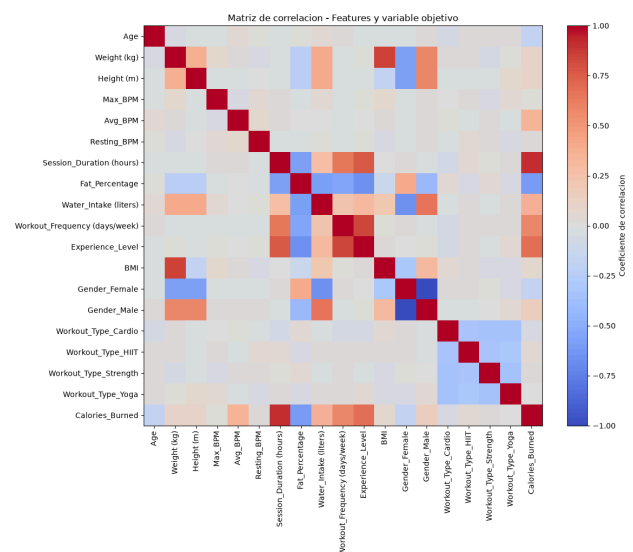


Figura 1. Matriz de correlación de las features del dataset

La matriz de correlación muestra que Session_Duration es, por amplio margen, la variable con mayor relación lineal con Calories_Burned, seguida de Experience_Level y Workout_Frequency. Esto es consistente con la intuición fisiológica de que entre más tiempo dura la sesión de entrenamiento, más calorías se queman.

Fat_Percentage presenta la correlación negativa más fuerte con el target, sugiriendo que a mayor porcentaje de grasa corporal, menor gasto calórico registrado en la sesión.

En contraste, variables como Max_BPM, Resting_BPM, BMI, Weight, Height y las categorías de Workout_Type tienen correlaciones cercanas a 0, por lo que aportan poca información lineal para predecir las calorías quemadas y fueron eliminadas del dataset del modelo.

También se detecta multicolinealidad relevante entre Workout_Frequency con Experience_Level y Gender_Male y Gender_Female lo cual indica que estas parejas de variables capturan información redundante y podría dejarse solo una de las dos.

Finalmente las features a utilizar para entrenar el modelo para predecir el feature Calories_Burned son los siguientes:

- Age
- Gender_Male
- Avg_BPM
- Session_Duration
- Fat_Percentage

- Water_Intake
- Experience_Level

Normalización: se normalizan las columnas numéricas (incluyendo la variable objetivo) al rango [0,1] porque el descenso de gradiente usa un único valor de alfa para actualizar todos los parámetros a la vez, y las variables originales tienen escalas muy distintas.

3. Carga

El dataset transformado, con las variables codificadas y la variable objetivo Calories_Burned ya definida, se guarda en un nuevo archivo listo para la etapa de modelado y entrenamiento del algoritmo de regresión lineal.

III. MODELO DE REGRESIÓN LINEAL

1. Cálculo de la hipótesis

En el contexto de la regresión lineal, la hipótesis se refiere a la función que utiliza el modelo para predecir el valor esperado de la variable dependiente (en este caso, las calorías quemadas) en función de las características de entrada. La fórmula general de la hipótesis para regresión lineal múltiple es la siguiente:

$$h(x) = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \theta_n x_n$$

Esta fórmula se implementó en una función que nos devuelve una predicción del número de calorías quemadas para un conjunto específico de características (edad, peso, duración de la sesión, frecuencia cardíaca, etc.).

2. Cálculo del error

El error del modelo, también conocido como función de pérdida o costo, mide la discrepancia entre las predicciones del modelo y los valores reales, es decir, mide qué tan precisas son las estimaciones del modelo.

En este trabajo se utilizó el Mean Squared Error (MSE), que se calcula tomando la diferencia entre cada predicción y su valor real correspondiente, elevando al cuadrado esa diferencia, sumando todas las diferencias cuadradas y dividiendo el resultado entre el número de muestras:

$$MSE = \frac{1}{n} \sum (Y_i - \hat{Y}_i)^2$$

El objetivo del entrenamiento es minimizar este error ajustando los parámetros a lo largo de las iteraciones.

3. Descenso de Gradiente

El descenso de gradiente es un algoritmo de optimización iterativo utilizado para entrenar modelos de aprendizaje automático, cuyo objetivo es minimizar una función de pérdida (en este caso, el MSE) ajustando progresivamente los parámetros del modelo.

En cada iteración, se calcula la derivada parcial de la función de costo con respecto a cada parámetro, y este parámetro se actualiza en la dirección opuesta al gradiente, con un tamaño de paso controlado por la tasa de aprendizaje.

$$\theta_j := \theta_j - \frac{\alpha}{m} \sum_{i=1}^m [(h_{\theta}(x_i) - y) x_i]$$

Primero se calcula la predicción del modelo para cada muestra, luego se obtiene el error restando el valor real. Después, se calcula la sumatoria del producto entre este error y el valor de cada feature correspondiente, y finalmente cada parámetro se actualiza restando este gradiente multiplicado por α . Este proceso se repite en cada época hasta que los parámetros dejan de cambiar de una iteración a otra, momento en el que se considera que el modelo alcanzó el mínimo de la función de costo.

Para este trabajo se utilizó una tasa de aprendizaje de $\alpha = 0.3$

4. Evaluación del modelo

Para evaluar el rendimiento del modelo se utilizó la métrica de R^2 (coeficiente de determinación), que mide qué proporción de la variabilidad de la variable objetivo puede ser explicada por las variables independientes del modelo. Cuanto más cercano a 1, mayor es la capacidad del modelo de explicar los datos.

$$R^2 = 1 - \frac{SS_{RES}}{SS_{TOT}} = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2}$$

Aquí se calcula la suma de los errores cuadrados del modelo y se compara contra la suma de las diferencias cuadradas entre cada valor real y la media de todos los valores reales.

5. División del dataset

Con el propósito de evaluar la capacidad del modelo para predecir datos no vistos, el dataset se dividió en tres partes: train con el 60% de los datos, validation con el 20%, y test con el 20% restante. El dataset se aleatorizó previamente para evitar sesgos derivados del orden original de las instancias.

El conjunto de train se usa para ajustar los parámetros del modelo. El conjunto de validation se evalúa después de cada época durante el entrenamiento, para monitorear la capacidad de generalización del modelo sin usar el conjunto de test. El conjunto de test se reserva exclusivamente para la evaluación final del modelo ya entrenado, proporcionando una métrica imparcial de su desempeño.

IV. RESULTADOS DEL ENTRENAMIENTO

El modelo se entrenó con descenso de gradiente por lote (batch gradient descent), usando una tasa de aprendizaje $\alpha = 0.3$ y un máximo de 10,000 epochs, deteniéndose antes si los parámetros dejaban de cambiar. En la ejecución, el entrenamiento consumió los 10,000 epochs completos.

1. Parámetros finales

Parámetro	Variable asociada	Valor
00	Bias	-0.0347
01	Age	-0.0975
02	Avg_BPM	0.2117
03	Session_Duration (hours)	0.7228
04	Fat_Percentage	-0.0058
05	Water_Intake (liters)	-0.0041
06	Experience_Level	-0.0042
07	Gender_Male	0.0610

Tabla 2. Parámetros finales del modelo

El coeficiente más grande en magnitud es el de Session_Duration, lo cual confirma que, incluso dentro del

modelo ya entrenado, esta variable domina la predicción, lo cual es coherente con su correlación encontrada previamente.

Avg_BPM es el segundo predictor más influyente, indicando que la intensidad cardiovascular de la sesión también aporta información más allá de la duración.

2. Curva de error de Train vs Validation

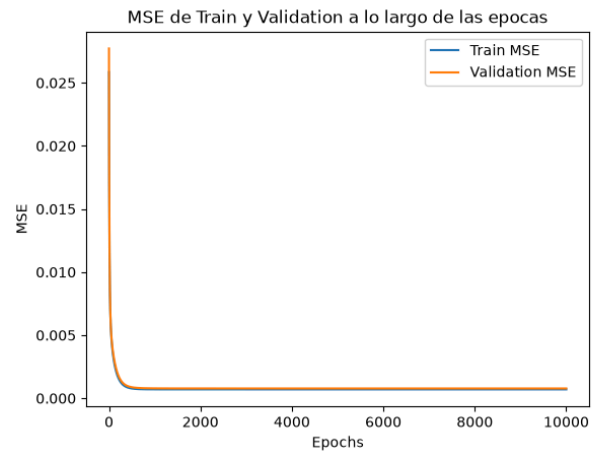


Figura 2. MSE de entrenamiento y validación a lo largo de las épocas.

El MSE cae muy rápidamente en los primeros epochs y luego se estabiliza en una meseta cercana a cero. Las curvas de train y validation prácticamente se superponen durante todo el entrenamiento, sin una separación creciente entre ambas, esta es la señal más clara de que no hay overfitting, ya que generaliza igual de bien a datos no vistos que a los datos con los que se entrenó.

3. Métricas finales

Conjunto	MSE normalizado	R ²
Train	0.0007	0.9796
Validation	0.0008	0.9778
Test	0.0007	0.9772

Tabla 3. Métricas finales del modelo

Los tres valores de R^2 son muy altos y extremadamente consistentes entre sí. Esto indica que el modelo explica aproximadamente el 97.7% de la varianza de las calorías quemadas usando únicamente 7 variables, y que su desempeño en datos nunca vistos (test) es prácticamente idéntico a su desempeño en train. El MSE, calculado sobre

la escala normalizada [0,1], es igualmente bajo y estable en los tres conjuntos. Por lo que el modelo lineal, resultó ser una buena aproximación para este problema.

4. Valores reales vs Predicciones

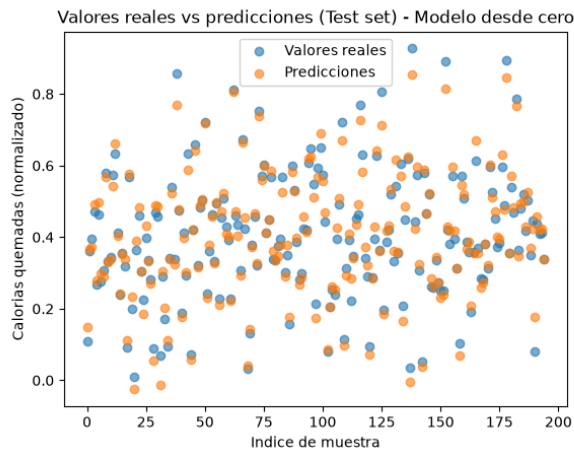


Figura 3. Comparación de valores reales y predicciones para cada muestra del conjunto de prueba.

Las dos nubes de puntos se superponen casi por completo a lo largo de todas las muestras del set de test, siguiendo la misma tendencia general de subida y bajada. No se observa sesgo, lo cual es consistente con el R^2 de 0.977 obtenido.

5. Distribución de residuos

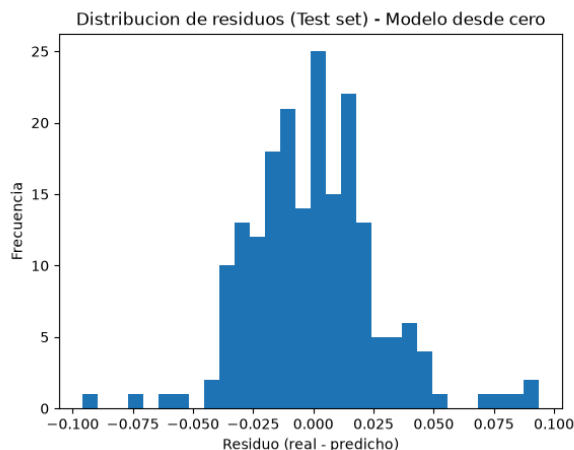


Figura 4. Histograma de residuos sobre el conjunto de prueba.

El histograma de residuos muestra una forma aproximadamente simétrica y centrada en cero, con la mayoría de los errores concentrados en un rango estrecho

alrededor de 0. Esta forma es cercana a una distribución normal centrada en cero, que es justamente el supuesto ideal para los errores de un modelo de regresión lineal.

6. Análisis de sesgo y varianza

Los tres valores de R^2 obtenidos son muy cercanos entre sí. Esta consistencia indica una varianza baja y que el modelo generaliza prácticamente igual de bien a datos que no vio durante el entrenamiento que a los datos con los que se entrenó, sin señales de overfitting. En cuanto al sesgo, dado que el modelo logra explicar cerca del 97.7% de la varianza de la variable objetivo con una relación lineal entre las features y las calorías quemadas, se puede considerar que tiene un sesgo bajo para este problema en particular. Esto sugiere que la relación entre las variables seleccionadas y las calorías quemadas es, en efecto, mayormente lineal.

V. IMPLEMENTACIÓN CON SCIKIT-LEARN UTILIZANDO RANDOM FOREST

Random Forest es un algoritmo de aprendizaje supervisado basado en ensambles, que puede utilizarse tanto para clasificación como para regresión. Su idea central es combinar las predicciones de múltiples árboles de decisión entrenados de forma independiente, en lugar de depender de un único modelo, con el objetivo de obtener una predicción más estable y con menor varianza que la de un árbol individual.

Cada árbol se entrena utilizando una técnica llamada bagging (también conocido como bootstrap aggregating), donde se generan múltiples subconjuntos de entrenamiento mediante muestreo aleatorio con reemplazo a partir del conjunto de datos original, y cada árbol se ajusta usando uno de estos subconjuntos. Adicionalmente, en cada split de un nodo, el algoritmo considera sólo un subconjunto aleatorio de las features disponibles en lugar de todas, lo cual introduce diversidad adicional entre los árboles y reduce la correlación entre ellos.

Para el caso de regresión, que es el que corresponde a este proyecto, la predicción final del bosque para una muestra determinada se obtiene promediando las predicciones individuales de todos los árboles que lo conforman. Este promedio suaviza el efecto de árboles individuales que puedan haberse “overfitteado” a su subconjunto de entrenamiento, lo que generalmente resulta en un modelo más robusto y con mejor capacidad de generalización que un solo árbol de decisión.

VI. PROCESO DE ENTRENAMIENTO

1. Hiperparámetros

La implementación se realizó utilizando la clase RandomForestRegressor de la biblioteca scikit-learn, manteniendo la mayoría de los hiperparámetros en sus valores por defecto para conservar la simplicidad del modelo. Los parámetros relevantes utilizados fueron los siguientes:

Parámetro	Valor	Descripción
n_estimators	100	Número de árboles de decisión que conforman el bosque.
random_state	42	Semilla fija para asegurar la reproducibilidad del muestreo aleatorio y de los resultados.

Tabla 4. Parámetros del entrenamiento con Random Forest

Las features de entrada, la variable objetivo y la división del dataset (60% entrenamiento, 20% validación, 20% prueba) se mantuvieron idénticas a las utilizadas en la implementación desde cero, con el fin de que ambos modelos sean comparables bajo las mismas condiciones. De igual forma, las variables numéricas se normalizaron al rango [0,1] utilizando MinMaxScaler.

2. Proceso de entrenamiento

A diferencia del modelo de regresión lineal, donde el entrenamiento consiste en ajustar iterativamente un vector de parámetros θ mediante descenso de gradiente, el proceso de entrenamiento de Random Forest sigue una lógica distinta.

Primero se generan 100 subconjuntos de entrenamiento mediante bootstrap a partir del conjunto de train. Para cada subconjunto se construye un árbol de decisión de regresión, donde cada nodo se divide buscando la variable y el punto de corte que minimizan el error cuadrático dentro de los subconjuntos resultantes, considerando en cada división sólo un subconjunto aleatorio de las features.

Después los árboles se expanden hasta que cada hoja es lo más pura posible en términos del criterio de error.

Y finalmente, una vez entrenados los 100 árboles, la

predicción del bosque para una nueva muestra se calcula como el promedio simple de las 100 predicciones individuales.

A diferencia del modelo lineal, este proceso no requiere definir un learning rate ni un número de epochs, ya que cada árbol se construye de una sola vez siguiendo un criterio de partición local, y la capacidad de generalización del bosque proviene de promediar múltiples modelos diversos en lugar de optimizar un único conjunto de parámetros de forma iterativa.

VII. RESULTADOS DEL ENTRENAMIENTO

1. Métricas finales

Conjunto	MSE normalizado	R ²
Train	0.0001	0.9962
Validation	0.0009	0.9747
Test	0.0007	0.9778

El modelo obtiene un R² de test de 0.9778, prácticamente idéntico al 0.9772 obtenido por el modelo de regresión lineal, lo que indica que Random Forest también logra explicar de gran forma la varianza de las calorías quemadas. Sin embargo, a diferencia del modelo lineal, aquí sí se observa una diferencia entre el desempeño en train (R² = 0.9963) y el desempeño en validation/test (0.9747-0.9778), lo cual se retoma en el análisis de sesgo y varianza.

2. Importancia de las features

Feature	Importancia
Session_Duration (hours)	0.8455
Avg_BPM	0.1017
Age	0.0219
Fat_Percentage	0.0165
Gender_Male	0.0081
Water_Intake (liters)	0.0058
Experience_Level	0.0005

Tabla 5. Importancia de las features según Random Forest.

A diferencia de los coeficientes de la regresión lineal, que indican tanto magnitud como dirección (positiva o negativa) de la relación con la variable objetivo, la importancia de features en Random Forest únicamente refleja qué tanto contribuye cada variable a reducir el error dentro de los árboles, sin indicar el sentido de la relación. Aun así, el

resultado es consistente con el modelo lineal y con el análisis de correlación inicial.

3. Valores reales vs Predicciones

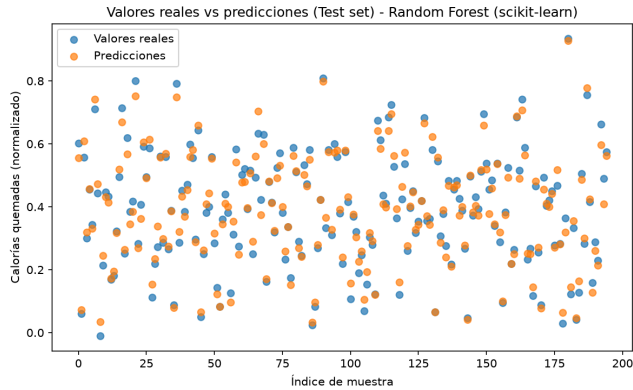


Figura 5. Comparación de valores reales y predicciones para cada muestra del conjunto de prueba en Random Forest

Al igual que en el modelo lineal, las nubes de puntos de valores reales y predicciones se superponen en gran medida a lo largo de las muestras del conjunto de prueba, siguiendo la misma tendencia general. No se aprecia un sesgo hacia sobreestimar o subestimar las calorías quemadas, lo cual es consistente con el R^2 de 0.977 obtenido.

4. Distribución de residuos

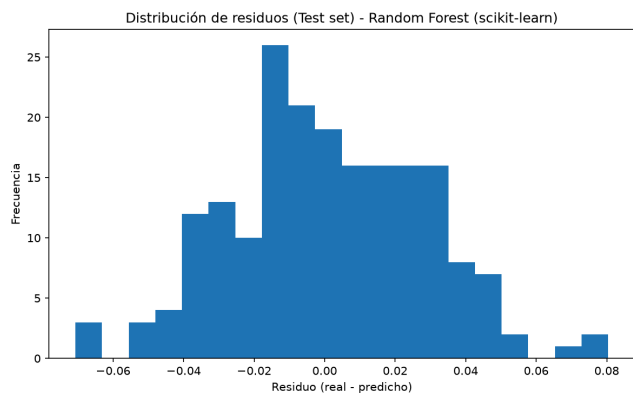


Figura 6. Histograma de residuos sobre el conjunto de prueba en Random Forest.

La distribución de residuos es aproximadamente simétrica y centrada en cero, con la mayoría de los errores concentrados en un rango estrecho alrededor de 0, de forma muy similar a lo observado en el modelo lineal.

Esto sugiere que Random Forest tampoco presenta un sesgo

relevante y que los errores restantes se deben en gran medida a ruido aleatorio del dataset, más que a una limitación estructural del modelo.

5. Análisis de sesgo y varianza

Se observa una brecha más marcada entre el desempeño en train ($R^2 = 0.9962$) y el desempeño en validation/test (0.9747-0.9778). Esta diferencia es indicio de una varianza más alta que la del modelo lineal.

Los árboles individuales tienden a ajustarse mucho a las particularidades del conjunto de entrenamiento, lo que produce un ligero overfitting que, aunque no es severo, sí es más pronunciado que en el modelo lineal.

En cuanto al sesgo, es bajo ya que el modelo es lo suficientemente flexible como para capturar relaciones no lineales e interacciones entre variables. Sin embargo, en este dataset dicha flexibilidad adicional no se traduce en una mejora relevante del desempeño, ya que la relación entre las features y la variable objetivo ya era mayormente lineal, como lo demuestra el modelo de regresión lineal hecho anteriormente.

VIII. ANÁLISIS COMPARATIVO ENTRE AMBOS MODELOS Y CONCLUSIONES

Mod.	Train MSE	Val. MSE	Test MSE	Train R^2	Val. R^2	Test R^2
RL	0.0007	0.0008	0.0007	0.9796	0.9778	0.9772
RF	0.0001	0.0009	0.0007	0.9962	0.9747	0.9778

Tabla 6. Comparación entre el modelo de regresión lineal y el modelo Random Forest

En términos de desempeño puro, ambos modelos son prácticamente equivalentes. El R^2 de test difiere en menos de un punto porcentual y el MSE es el mismo.

Esto indica que, para este problema y con las features seleccionadas, la relación entre las variables predictoras y las calorías quemadas es mayormente lineal, por lo que la capacidad adicional de Random Forest para modelar relaciones no lineales e interacciones complejas no se traduce en una ganancia relevante de desempeño.

La regresión lineal resulta ser la alternativa más eficiente para este caso de uso, ya que alcanza un desempeño

prácticamente idéntico al de Random Forest, con una varianza menor, una interpretabilidad mucho mayor gracias a sus coeficientes y un costo computacional considerablemente menor.

Es importante señalar que los valores de R^2 obtenidos por ambos modelos de 0.977 y 0.978 son inusualmente altos para un problema de estimación de gasto calórico basado en datos fisiológicos.

Como prueba adicional, se entrenó un modelo de regresión lineal utilizando únicamente la variable Session_Duration, obteniendo por sí sola un R^2 de prueba de 0.829. Que una sola variable explique más del 80% de la varianza de las calorías quemadas sugiere que el dataset utilizado no proviene de mediciones reales de dispositivos, sino que fue generado de forma sintética a partir de una fórmula o relación predefinida.

El desempeño obtenido refleja meramente qué tan bien cada modelo recupera la relación subyacente del dataset, y no necesariamente qué tan bien generalizaría a mediciones reales de gasto calórico en un entorno de gimnasio, ya que ahí entrarían muchísimas más variables en juego.

IX. ANEXOS

<https://github.com/Emiliano1410/BurnedCalories-Prediction-Model>

X. REFERENCIAS

[1] IBM, "What is random forest?" IBM.com. [En línea]. Disponible en: <https://www.ibm.com/think/topics/random-forest>

[2] IBM, "What is linear regression?" IBM.com. [En línea]. Disponible en: <https://www.ibm.com/think/topics/linear-regression>

[3] IBM, "What is bagging?" IBM.com. [En línea]. Disponible en: <https://www.ibm.com/think/topics/bagging>

[4] IBM, "What is overfitting?" IBM.com. [En línea]. Disponible en: <https://www.ibm.com/think/topics/overfitting>

[5] IBM, "What is gradient descent?" IBM.com. [En línea]. Disponible en: <https://www.ibm.com/think/topics/gradient-descent>

[6] scikit-learn developers, "1.11. Ensembles: Gradient boosting, random forests, bagging, voting, stacking — scikit-learn documentation." [En línea]. Disponible en: <https://scikit-learn.org/stable/modules/ensemble.html>

[7] scikit-learn developers, "sklearn.ensemble.RandomForestRegressor — scikit-learn documentation." [En línea]. Disponible en: <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>

[8] scikit-learn developers, "sklearn.linear_model.LinearRegression — scikit-learn documentation." [En línea]. Disponible en: https://scikit-learn.org/stable/modules/generated/sklearn.linear_model.LinearRegression.html

[9] scikit-learn developers, "sklearn.preprocessing.MinMaxScaler — scikit-learn documentation." [En línea]. Disponible en: <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.MinMaxScaler.html>

[10] Wikipedia, "Random forest." [En línea]. Disponible en: https://en.wikipedia.org/wiki/Random_forest

[11] Wikipedia, "Linear regression." [En línea]. Disponible en: https://en.wikipedia.org/wiki/Linear_regression

[12] Wikipedia, "Gradient descent." [En línea]. Disponible en: https://en.wikipedia.org/wiki/Gradient_descent

[13] Wikipedia, "Coefficient of determination." [En línea]. Disponible en: https://en.wikipedia.org/wiki/Coefficient_of_determination

[14] Wikipedia, "Bias–variance tradeoff." [En línea]. Disponible en: https://en.wikipedia.org/wiki/Bias%E2%80%93variance_tradeoff

[15] GeeksforGeeks, "Bias-Variance Trade Off - Machine Learning." [En línea]. Disponible en: <https://www.geeksforgeeks.org/machine-learning/bias-vs-variance-in-machine-learning/>

[16] GeeksforGeeks, "Random Forest Regression in Python." [En línea]. Disponible en: <https://www.geeksforgeeks.org/machine-learning/random-forest-regression-in-python/>

[17] V. Khorasani, "Gym Members Exercise Dataset," Kaggle, 2024. [En línea]. Disponible en: <https://www.kaggle.com/datasets/valakhorasani/gym-members-exercise-dataset>